UNIVERSIDAD NACIONAL DEL SUR
Departamento de Ciencias e Ingeniería de la Computación
Compiladores e Intérpretes — Segundo Cuatrimestre de 2026

Compilador de MiniJava

Etapa 1 — Analizador Léxico

y Módulo Principal

Agustín Paul

Informe de entrega

Implementación por *Método por Estado* (patrón State).
Los 56 componentes léxicos, sus expresiones regulares y sus autómatas.

29 de agosto de 2026

Índice

1. Alcance **1**
- 1.1. Qué hace el programa . . . 1
- 1.2. Cómo compilarlo y ejecutarlo . . . 1

2. Notación de expresiones regulares **1**

3. Los 56 componentes léxicos **2**
- 3.1. Identificadores . . . 2
- 3.2. intLiteral . . . 3
- 3.3. charLiteral . . . 3
- 3.4. litString . . . 4
- 3.5. floatLiteral . . . 4
- 3.6. El escape Unicode . . . 5
- 3.7. Puntuación . . . 5
- 3.8. Operadores . . . 6
- 3.9. EOF . . . 6
  - 3.9.1. Cuándo se da por terminado el archivo . . . 6
  - 3.9.2. En qué línea cae el EOF . . . 7
- 3.10. Lo que no es un token . . . 7
  - 3.10.1. Separadores . . . 7
  - 3.10.2. Comentarios . . . 7

4. Los autómatas **8**
- 4.1. Identificadores y palabras clave . . . 8
- 4.2. charLiteral . . . 8
- 4.3. litString . . . 8
- 4.4. floatLiteral . . . 9
- 4.5. Comentario multilínea . . . 10
- 4.6. El reparto entre autómata y tabla . . . 10

5. Decisiones tomadas **10**

6. Manejo de errores **11**
- 6.1. Formato del reporte . . . 11
- 6.2. Criterio de recuperación . . . 12
- 6.3. Por qué el charLiteral no resincroniza . . . 12
- 6.4. Dónde apunta el ^ . . . 13
- 6.5. Ubicar el error sin releer el archivo . . . 13

7. Logros perseguidos **14**

8. Estrategia de prueba **14**
- 8.1. Cómo se declara el oráculo de un caso . . . 14

9. Divergencias deliberadas respecto de Java **15**

1 Alcance

Este informe documenta la Etapa 1 del compilador de MiniJava: el **Analizador Léxico** y el **Módulo Principal** que lo consume por consola. Cubre la expresión regular de los 56 componentes léxicos sin omitir ninguno, los autómatas que los reconocen, las decisiones tomadas donde la especificación admitía más de una lectura, el tratamiento de errores y los ocho logros perseguidos.

1.1 Qué hace el programa

Toma un archivo fuente MiniJava y emite por salida estándar la secuencia de sus componentes léxicos, uno por línea, en el formato `(NombreToken,Lexema,NroLinea)`. Si no hubo errores, cierra con `[SinErrores]`; si los hubo, emite un reporte por cada uno, terminado en el código de contrato `[Error:Lexema|NroLinea]`.

```
$ java -jar Compilador.jar programa1.java
(litString,"hola",1)
(idMV,v1,2)
(op+,+,2)
(idMV,chau,2)
(pr_if,if,3)
(pr_class,class,3)
(llaveC,},3)
(EOF,$,3)

[SinErrores]
```

El renglón en blanco antes de `[SinErrores]` — y el equivalente antes de `[Error:...]` — son los de los ejemplos de *Pautas Etapa 1*. El archivo fuente puede tener cualquier extensión: el compilador no inspecciona el nombre, sólo abre la ruta que recibe.

1.2 Cómo compilarlo y ejecutarlo

```
cd compilador
mvn package                                    # genera target/Compilador.jar
java -jar target/Compilador.jar programa1.java

mvn test        # suite propia + los 2 testers de la cátedra, en la misma pasada
mvn verify      # además exige los umbrales de cobertura
```

Requiere Java 25 (`maven.compiler.release=25`).

2 Notación de expresiones regulares

Toda expresión regular de este informe usa la notación siguiente. La cardinalidad se escribe siempre como **exponente** del símbolo o del grupo al que afecta. Los dos operadores que más conviene distinguir son la estrella de Kleene, que admite *cero o más* repeticiones, y la repetición acotada X^{m-n}, que es la que necesita el literal entero.

Forma	Significado
`X*`	cero o más repeticiones de `X` (estrella de Kleene)
`X+`	una o más repeticiones de `X`
`X?`	cero o una repetición de `X`
`Xm-n`	entre `m` y `n` repeticiones, **ambos extremos inclusive**
`A | B`	alternativa: `A` o `B`
`( ...)c`	agrupación; el exponente `c` afecta al grupo entero

Y las clases de caracteres que se usan como abreviatura:

```
Digito     = [0-9]
LetraMayus = [A-Z]
LetraMin   = [a-z]
Letra      = LetraMayus | LetraMin
Hex        = [0-9a-fA-F]
```

> Una precisión sobre "letra" y "dígito": la implementación los reconoce con `Character.isLetter` y `Character.isDigit`, que abarcan más que el ASCII. Las consecuencias están en la sección 9.

3 Los 56 componentes léxicos

Familia	Cant.	Tokens
Palabras clave	21	pr_class, pr_extends, pr_interface, pr_implements, pr_static, pr_boolean, pr_char, pr_int, pr_void, pr_public, pr_if, pr_else, pr_while, pr_return, pr_var, pr_this, pr_new, pr_null, pr_true, pr_false, pr_float
Identificadores	3	idClase, idGen, idMV
Literales	4	intLiteral, charLiteral, litString, floatLiteral
Puntuación	10	parenA, parenC, llaveA, llaveC, corcheteA, corcheteC, puntoY-Coma, coma, punto, dosPuntos
Operadores	17	op_mayor, op_menor, op_negacion, op_asignacion, op_igual, op_mayorIgual, op_menorIgual, op_distinto, op_and, op_or, op_modulo, op+, op-, op_mult, op_div, op_incremento, op_decremento
Fin de archivo	1	EOF
Total	**56**	

Los nombres de la columna derecha son literales: son exactamente los que se imprimen, porque el formato de salida fijado es `(NombreToken,Lexema,NroLinea)`.

3.1 Identificadores

Los tres se distinguen por la caja de su primera letra y, entre los dos que empiezan con mayúscula, por su longitud.[1]

[1] **Notación.** `X*` = cero o más repeticiones de `X` (estrella de Kleene); `X+` = una o más; `X?` = cero o una; `X1-9` = entre 1 y 9 repeticiones, *ambos extremos inclusive*; `A | B` = alternativa. La tabla completa está en la sección 2.

Token	Expresión regular	Ejemplos
idGen	`LetraMayus`	`A`, `T`
idClase	`LetraMayus ( Letra | Digito | _ )+`	`Alpha`, `A1`, `Mi_Clase`
idMV	`LetraMin ( Letra | Digito | _ )*`	`v1`, `x`, `mi_metodo`

idGen y idClase comparten el primer carácter y se distinguen por longitud: **una** mayúscula sola es idGen; una mayúscula seguida de al menos un carácter más es idClase. Ahí está la diferencia entre los dos exponentes de la tabla: el $^+$ de idClase exige ese carácter adicional, mientras que el * de idMV admite el identificador de una sola letra.

Un identificador que empieza con minúscula es siempre idMV, **salvo** que su lexema completo coincida con una de las 21 palabras clave; ver la sección 4.6.

3.2 intLiteral

```
intLiteral = Digito^1-9
```

Entre uno y nueve dígitos, ambos extremos inclusive. La cota superior es parte de la especificación del lenguaje, no una limitación de la implementación, y por eso la repetición es **acotada** y no una estrella de Kleene: `Digito*` describiría un lenguaje distinto —y además admitiría el literal vacío—. Escrita sin la abreviatura de cardinalidad:

```
intLiteral = Digito Digito? Digito? Digito? Digito? Digito? Digito? Digito?
```

El autómata que le corresponde es, por lo tanto, una **cadena de nueve estados terminales sin ciclo**: un bucle sobre un único estado aceptaría corridas de cualquier longitud, que es justamente lo que la cota prohíbe. Cada uno de los nueve estados es de aceptación, porque un literal entero puede terminar en cualquier cantidad de dígitos entre uno y nueve; el décimo dígito no tiene transición, y ahí está el error.



El límite se valida al final de la corrida, no durante. El estado consume la corrida *completa* de dígitos y recién al terminar comprueba que no supere nueve. La razón es que si el carácter siguiente a la corrida es un `.` o una `e`/`E`, la corrida continúa hacia un floatLiteral, que **no** tiene ese límite; cortar en el noveno dígito rompería esa continuación. Por eso `1234567890.5` es un float válido con diez dígitos en la parte entera, mientras que `1234567890` solo es un error.

3.3 charLiteral

```
charLiteral = ' X '  |  ' \ Y '  |  ' \ U '

X = cualquier carácter salvo \ y '
Y = cualquier carácter salvo u
U = escape Unicode
```

La especificación da aquí una **producción con dos alternativas**, y las variables `X` e `Y` son independientes entre sí. Eso hace que la barra invertida y el carácter que la sigue formen una *unidad*: de ahí que `'\\'` y `'\"` sean literales válidos. El literal vacío `''` no está en ninguna alternativa y es un error.

Como `Y` es "cualquier carácter", `'\q'` es válido, con valor `q`: una permisividad explícita de la especificación, más laxa que Java. Y "cualquier carácter" se toma al pie de la letra: valen las letras acentuadas, la eñe y los alfabetos no latinos igual que las de ASCII. Las dos únicas

excepciones no son restricciones del juego de caracteres sino estructurales —un salto de línea o un fin de archivo dejan el literal sin cerrar—, y la `u` es el único carácter que no se consume como común, porque abre el escape Unicode.

Ésta es la primera de **dos** asimetrías entre el charLiteral y el litString. La segunda, la de recuperación ante un error, está en la sección 6.3, y las dos salen del mismo lugar: la especificación describe cada literal con una clase de regla distinta.

3.4 litString

```
litString = " ( Z | \ C )* "    con la " de cierre NO antecedida por \

Z = cualquier carácter salvo \, " y salto de línea
C = cualquier carácter salvo salto de línea  (incluye \ y ")
```

Por qué esta ER tiene otra forma que la del charLiteral. La especificación *no* da una producción para el string. Da una regla **posicional**:

> *"es posible que en la secuencia de caracteres aparezca una comilla doble siempre y cuando se la anteceda por el caracter \. Por esto último la comilla doble que marca el final de un string correcto nunca estará antecedida por ese caracter."*

Es un *lookbehind* de un carácter sobre el texto crudo. Consecuencia directa: **no existe un conjunto de escapes válidos** dentro de un litString. `"hola \q"` es un string común y corriente, en el que la barra y la `q` son dos caracteres cualesquiera.

Fuente	Resultado	Por qué
`"hola \q"`	litString válido	la " de cierre no está antecedida por \
`"di \"hola\" chau"`	**un solo** litString	las dos " internas están antecedidas por \
`"a\\"`	*sin cerrar*	la " final está antecedida por \: no cierra
`"a\\\""`	litString válido	la primera " no cierra, la segunda sí
`"barra \\ al medio"`	litString válido	la barra sólo importa pegada a la comilla de cierre

La tercera fila es el único punto en que esta regla se aparta de Java, donde `"a\\"` denota la barra invertida: aquí una barra no "cancela" a la anterior, porque la regla no habla de pares sino de qué carácter antecede a la comilla de cierre. Se resolvió a favor de la especificación escrita, que es explícita en decir *nunca*.

Un litString tampoco puede contener un salto de línea real: si aparece uno antes de la comilla de cierre, el literal quedó sin cerrar.

3.5 floatLiteral

```
floatLiteral = Digito+ . Digito* Exponente? Sufijo?
             | Digito+ Exponente Sufijo?
             | Digito+ Sufijo

Exponente = ( e | E ) ( + | - )? Digito+
Sufijo    = f | F | d | D
```

Misma estructura que los floats de Java, con la salvedad que fija la consigna: **siempre empiezan con un dígito**. Por eso `.5` no es un float en MiniJava, sino el token punto seguido de intLiteral(5).

La cardinalidad del exponente, que es donde se juega todo. [2] En `Exponente = ( e | E ) ( + | - )? Digito+` conviven los tres casos, y cada uno dice algo distinto:

`( e | E )` la marca de exponente es obligatoria y va *sin* exponente de cardinalidad: aparece exactamente una vez.

`( + | - )?` el signo es **opcional** — cero o una vez. `1e5`, `1e+5` y `1e-5` son los tres válidos.

`Digito+` los dígitos del exponente son **uno o más**, *nunca* `Digito*`. Ésta es la diferencia que hace que `1e` y `1.5e+` sean errores: con `*` el exponente vacío sería válido y `1e` tokenizaría como un float.

En cambio, la parte fraccionaria sí lleva estrella de Kleene: `Digito+ . Digito*` admite `1.` como float válido, con cero dígitos después del punto, igual que en Java.

Ninguna de las tres partes tiene cota superior de dígitos, a diferencia del intLiteral. Un float fuera del rango representable (`3.5e38f`, `1e999`) es un literal léxicamente **válido**: en Java el rechazo por rango es un chequeo semántico, no léxico, y aquí se respeta esa separación de fases.

No hay retroceso a un token más corto: una vez consumida la `e` como inicio de exponente, el literal está comprometido, y si no vienen dígitos es un error en lugar de un intLiteral seguido de un idMV.

3.6 El escape Unicode

`U = \ u+ Hex Hex Hex Hex` = `\ u+ Hex4-4`

El marcador `u` se repite **una o más** veces: `\u0041`, `\uu0041` y `\uuuu0041` son todos válidos y equivalentes. Es semántica heredada de Java, que la necesita para poder representar un `\u` literal sin re-traducirlo. Después de la última `u` se exigen exactamente cuatro dígitos hexadecimales — no más, no menos: no es `Hex+` ni `Hex*`, es una repetición acotada de extremos iguales, `Hex4-4`.

El mismo escape sirve dentro de un charLiteral y dentro de un litString, y los estados que lo reconocen están parametrizados por el estado al que deben volver.

3.7 Puntuación

Token	ER	Token	ER	Token	ER	Token	ER
parenA	(	parenC	)	llaveA	{	llaveC	}
corcheteA	[	corcheteC	]	puntoYComa	;	coma	,
punto	.	dosPuntos	:				

Los diez son tokens de un solo carácter, sin lookahead: la ER de cada uno es ese carácter y nada más.

[2] **Notación.** `X*` = cero o más repeticiones de `X` (estrella de Kleene); `X+` = una o más; `X?` = cero o una; `X1-9` = entre 1 y 9 repeticiones, *ambos extremos inclusive*; `A | B` = alternativa. La tabla completa está en la sección 2.

3.8 Operadores

Token	ER	Token	ER
op_mayor	>	op_mayorIgual	>=
op_menor	<	op_menorIgual	<=
op_negacion	!	op_distinto	!=
op_asignacion	=	op_igual	==
op+	+	op_incremento	++
op-	-	op_decremento	-
op_mult	*	op_div	/
op_modulo	%		
op_and	&&	op_or	||

Se agrupan en tres clases por su comportamiento en el autómata:

Con sufijo opcional (`>`, `<`, `!`, `=`, `+`, `-`): el primer carácter ya forma un token válido y el segundo lo extiende. El estado que los reconoce es **terminal**: si el sufijo no viene, emite el token corto sin consumir el carácter que miró.

Con segundo carácter obligatorio (`&&`, `||`): `&` y `|` sueltos **no** son tokens del lenguaje. El estado es **no terminal**: si el segundo carácter no viene, es un error léxico.

De un solo carácter (`*`, `%`, y `/` cuando no abre un comentario).

3.9 EOF

Es un token más, con lexema `$`, tal como lo fija la especificación. Su ER es trivial: un solo símbolo sobre la condición de fin de contenido.

El lexema `$` y el centinela de fin de archivo son cosas distintas. El centinela que devuelve el manejador de archivo cuando se acabó el contenido es `(char) 26`, valor fijado por la interfaz `SourceManager`. El `$` es solamente el lexema que se *imprime* para el token `EOF`: no se lee del fuente y no tiene ninguna relación con el centinela. Un `$` escrito en el fuente es un símbolo inválido y produce un error léxico.

3.9.1 Cuándo se da por terminado el archivo

`SourceManager` transporta por un mismo canal dos cosas de naturaleza distinta: los caracteres del fuente y la señal de que los caracteres se terminaron, porque `getNextChar()` devuelve un `char` y el centinela también es un `char`. Eso funcionaría si el valor centinela nunca pudiera aparecer como dato, pero `0x1A` es un byte perfectamente legítimo dentro de un archivo. Recibir el centinela es, entonces, condición **necesaria pero no suficiente**.

Por eso el fin de archivo se decide con una **conjunción**, y la interfaz se extiende con `estaEnFinDeArchivo()` para aportar el segundo término:

$$\text{fin de archivo} \iff \text{llegó el centinela} \ \wedge \ \text{la posición de lectura confirma que se acabó el contenido}$$

Si llega el centinela pero la posición dice que todavía queda archivo, entonces no era un fin de archivo: es un carácter del fuente y se reporta como símbolo inválido. Sin ese segundo

término, un `0x1A` embebido cortaría el análisis en silencio, informando `[SinErrores]` sobre un fuente que nunca se terminó de leer.

En la implementación aparece un solo chequeo, porque el segundo término *implica* al primero: el manejador devuelve el centinela en una única rama, que es exactamente la misma en la que registra que se pasó del final. La conjunción se colapsa y el $\wedge$ queda implícito; en los estados se testea sólo el término que discrimina. La implicación no vale al revés, y ahí está todo el asunto: un `0x1A` embebido cumple el primer término y no el segundo.

3.9.2 En qué línea cae el EOF

$$\text{línea del EOF} \;=\; \text{cantidad de saltos de línea consumidos} \;+\; 1$$

Dicho de manera más operativa: se arranca en la línea 1 y se suma 1 cada vez que se consume un `\n`; la línea en la que se queda parado al agotar el contenido es la del EOF.

El caso que conviene mirar dos veces es el del archivo que **termina** en salto de línea. Un `\n` no es "el final de una línea" sino un *separador*, así que después del último salto hay una línea más —vacía, pero existe— y es ahí donde cae el EOF. La consecuencia práctica es que ese salto final tiene que entregarse al léxico como cualquier otro carácter: un manejador que lo omita reporta el EOF una línea antes de lo que corresponde.

La fórmula es una sola y cubre los dos casos, como muestran los tres archivos de ejemplo provistos:

Archivo	Saltos	¿Termina en salto?	Línea del EOF
`lexSinErrores01.java`	21	sí	22
`lexSinErrores02.java`	12	no	13
`lexSinErrores03.java`	12	sí	13

3.10 Lo que no es un token

3.10.1 Separadores

Espacio, tabulador y salto de línea. Se descartan sin agregarse a ningún lexema. LF, CR y CRLF cuentan cada uno como **un** solo salto de línea, de modo que dos archivos que difieran únicamente en el convenio de fin de línea reportan los mismos números de línea.

El *form feed* y el tabulador vertical **no** son separadores en MiniJava, aunque sí lo son en Java: no están en la lista de la especificación, y por lo tanto producen error léxico.

3.10.2 Comentarios

De línea (`// ...` hasta el fin de línea) y multilínea (`/* ...*/`). Se descartan por completo: no producen token y su contenido no forma parte de ningún lexema. **Todo** lo que está dentro de un comentario se ignora, incluidos escapes Unicode mal formados y comillas sin cerrar.

La ER del comentario multilínea es la estándar, la misma que usan `lex`/`flex` para los comentarios de C:

```
/\*  ( [^*] | \*+ [^*/] )*  \*+  /
```

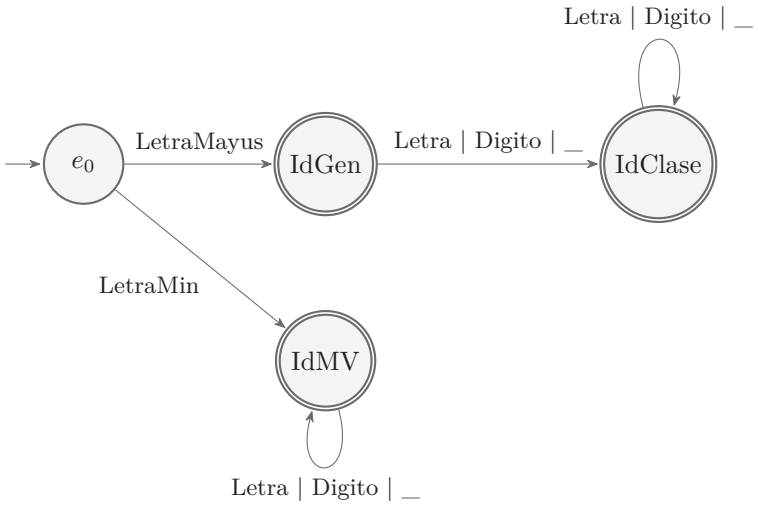
En palabras: tras el `/*`, el contenido es cualquier cantidad de (un carácter que no sea `*`) o (uno o más `*` seguidos de un carácter que no sea ni `*` ni `/`), y cierra con uno o más `*` seguidos de `/`. Los comentarios **no anidan**.

Un comentario multilínea sin cerrar es un error léxico, y se reporta con el lexema `/*` y la línea de **apertura**, no la del fin de archivo donde se descubrió.

4 Los autómatas

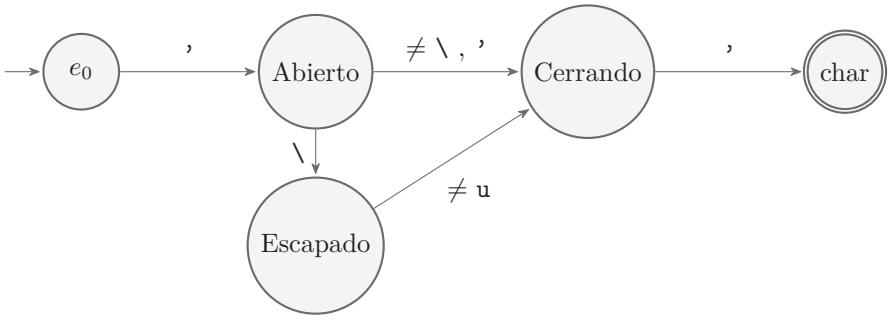
La implementación usa la técnica de *Método por Estado*: hay una clase por estado del autómata finito, cada una responsable de una sola transición. El driver es iterativo — cada estado devuelve o bien el estado siguiente, o bien el token a emitir — de modo que la profundidad de pila no crece con la longitud del lexema.

4.1 Identificadores y palabras clave



El estado IdGen es **terminal**: si el token termina ahí, es idGen; si aparece un carácter más, transiciona a IdClase. Al emitir desde IdMV se consulta la tabla de lexemas: si el lexema completo coincide con una palabra reservada, se emite la palabra clave.

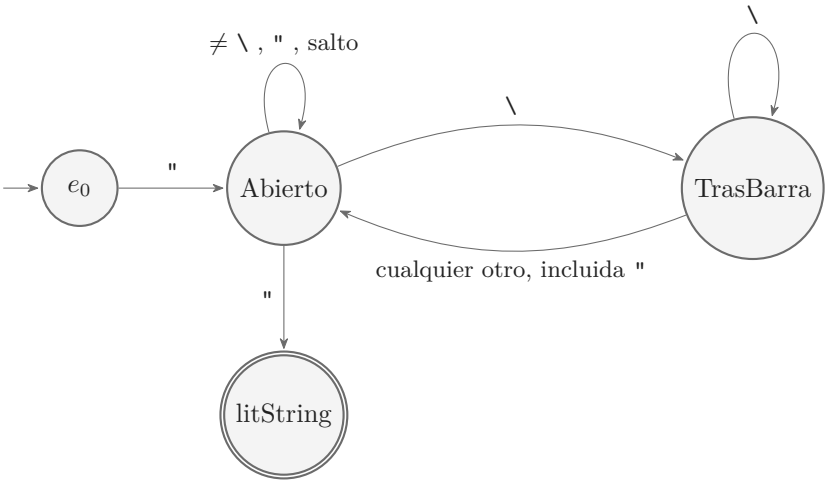
4.2 charLiteral



Desde Escapado, una `u` deriva al autómata del escape Unicode, que vuelve a Cerrando tras consumir los cuatro dígitos hexadecimales. Los tres caminos que no llegan al estado final son los tres errores del charLiteral: `''` (vacío), un fin de línea o de archivo antes de cerrar (*sin cerrar*), y cualquier otro carácter donde debía ir la comilla de cierre (*mal formado*).

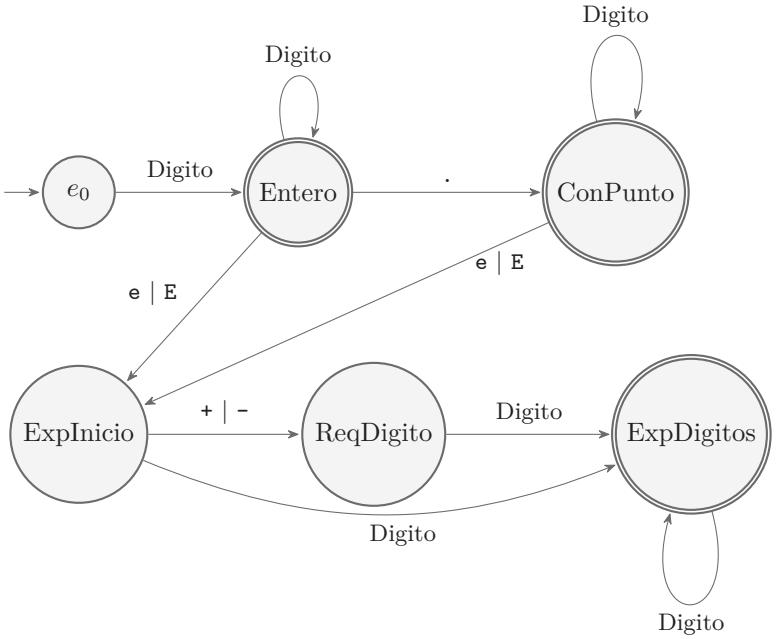
4.3 litString

El *lookbehind* de la sección 3.4 es un bit de memoria, y se modela con dos estados: en Abierto el carácter anterior no fue una barra; en TrasBarra sí lo fue.



Hay que reparar en el bucle de TrasBarra sobre \: una barra que sigue a otra barra **no** devuelve al estado Abierto, porque la comilla que venga después seguirá estando antecedida por una barra. Eso es exactamente lo que hace que `"a\\"` quede sin cerrar. Desde TrasBarra, una `u` deriva al escape Unicode.

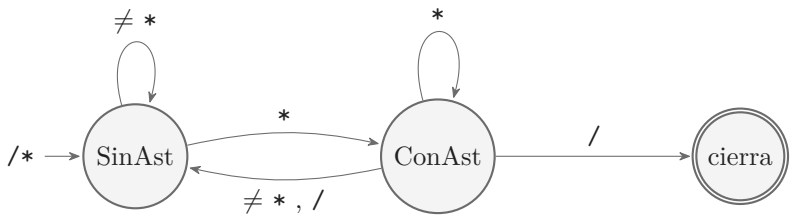
4.4 floatLiteral



Entero, ConPunto y ExpDigitos son terminales; ExpInicio y ReqDigito no lo son, y ahí está representado el `Digito`$^+$ del exponente: llegar a ExpInicio y no encontrar un dígito ni un signo es el error de `1e`. Los sufijos `f`/`F`/`d`/`D` pueden cerrar desde Entero, ConPunto o ExpDigitos; se omiten del diagrama para no saturarlo.

Los bucles sobre Entero, ConPunto y ExpDigitos son correctos acá y no contradicen la cadena de nueve estados del intLiteral: ninguna de las tres partes de un floatLiteral tiene cota de dígitos, así que la repetición *sí* es una estrella de Kleene. La cota de nueve pertenece únicamente al literal entero.

4.5 Comentario multilínea



4.6 El reparto entre autómata y tabla

De los 56 tokens, **49 tienen lexema fijo**, en el sentido de que el tipo y el lexema se determinan mutuamente: las 21 palabras clave, los 10 símbolos de puntuación, los 17 operadores y EOF. Los otros 7 — los tres identificadores y los cuatro literales — leen su lexema del fuente.

Los 49 no necesitan un estado que lleve su tipo encima: alcanza con que el autómata delimite el lexema y con consultar una tabla. Lo único que la tabla *no* puede resolver es el *maximal munch*: no sabe si un > es op_mayor o el principio de un op_mayorIgual, y eso lo decide el autómata mirando el carácter siguiente. El reparto queda nítido:

> **el autómata delimita el lexema; la tabla lo nombra.**

Es la técnica recomendada para las palabras clave, generalizada a los 49. Modelar cada palabra reservada con estados dedicados exigiría casi un estado por letra, fusionados con los estados de identificador: decenas de estados frágiles para algo que una tabla resuelve en tiempo constante.

5 Decisiones tomadas

Dos reglas gobiernan todas las decisiones del proyecto:

1. **Lo que la especificación fija, se respeta al pie de la letra.**
2. **Lo que no fija, se resuelve como en Java** — regla escrita explícitamente en la consigna.

Los casos que admitían más de una lectura, y cómo se resolvió cada uno:

Caso	Decisión	Fundamento
`float` y `double` como palabras reservadas	`float` es reservada; `double` **no**: sigue siendo un idMV común	`float` se agregó por confirmación directa de la cátedra; que MiniJava tenga un único tipo de punto flotante no convierte a `double` en reservada
Gramática del floatLiteral	la de Java, con la salvedad de que siempre empieza con dígito	la consigna la describe en una sola frase: “la misma estructura que los floats de Java”
Límite de dígitos de los floats	sin límite, a diferencia del intLiteral	la cota de 9 dígitos está enunciada sólo para el literal entero
Float fuera de rango (`1e999`)	literal léxicamente válido	en Java el rechazo por rango es un chequeo *semántico*; el léxico no valida magnitudes

Caso	Decisión	Fundamento
Escapes en charLiteral	cualquier carácter tras la barra	la especificación da una producción con dos alternativas y "cualquier caracter" explícito
Escapes en litString	no hay conjunto de escapes: cierra la primera " no antecedida por \	la especificación da una regla *posicional*, no una producción; ver sección 3.4
& y | sueltos	error léxico	la lista de operadores incluye && y ||, y no los incluye sueltos
"" (char vacío)	error léxico	no está en ninguna de las dos alternativas de la producción
Marcador u repetido	\uu0041 es válido y equivale a \u0041	semántica de Java (JLS §3.3), verificada contra la especificación
Maximal munch	se consume siempre el token más largo posible, sin retroceso	comportamiento estándar; 1234abc da intLiteral(1234) y idMV(abc)
Escape Unicode mal formado en un litString	se reporta y se descarta el literal hasta su comilla de cierre, sin emitir token	sobre "a\u12G4" + v1 se emite un error y luego op+ e idMV(v1): el literal roto no se emite, y el análisis retoma en el token siguiente
Rango de un intLiteral	la cota es de *dígitos*, no de valor: 0000000001 es error	la especificación acota dígitos, no magnitud

6 Manejo de errores

6.1 Formato del reporte

Ante un error, el léxico lanza una excepción que el Módulo Principal captura y formatea. La última línea es siempre el código de contrato `[Error:Lexema|NroLinea]`; las anteriores son el reporte legible, que es uno de los logros perseguidos.

```
(idMV,v1,2)
Error Léxico en línea 2, columna 6: # no es un símbolo válido
Detalle: v1 + # chau
             ^

[Error:#|2]
(idMV,chau,2)
```

El lexema reportado es siempre el armado hasta el momento del error, como exige la consigna, y el código de contrato lleva ese mismo lexema. Si el carácter ofensivo *es* el problema, queda incluido en el lexema; si el problema es el lexema ya construido (un & sin su par, un entero de más de nueve dígitos, un exponente sin dígitos), el carácter que se está mirando es un token legítimo que no tiene nada que ver con el error, y por lo tanto no se consume ni se incluye.

Los lexemas de error nunca llevan un carácter invisible crudo: un salto de línea dentro de un lexema partiría la línea del reporte en dos y rompería el formato de un elemento por línea. Se renderizan con su escape (\n, \t, \u00a0).

6.2 Criterio de recuperación

> **Todo estado que lanza un error consume del fuente exactamente los caracteres que reporta en el lexema, ni uno más.**

No hay un paso de recuperación posterior: el siguiente pedido de token arranca en el primer carácter que **no** formó parte del lexema reportado. Esto coincide con lo que hace `javac`, que tampoco descarta caracteres ajenos al lexema ofensivo, y garantiza estructuralmente la terminación: el lexema de un error nunca es vacío, así que cada error consumió al menos un carácter y la posición de lectura siempre avanza.

La única excepción es el escape Unicode mal formado dentro de un litString, que además descarta lo que queda del literal hasta su comilla de cierre. Es recuperación en modo pánico con token de sincronización: el delimitador es la comilla de cierre, y el estado conocido al que se vuelve es "fuera de todo literal". Sin resincronizar, la comilla de *cierre* abriría un literal nuevo que se comería el resto de la línea.

6.3 Por qué el charLiteral no resincroniza

Que el litString descarte hasta su comilla de cierre y el charLiteral no descarte nada no es una inconsistencia: es que el error deja al analizador en lugares distintos.

> Cuando un litString falla, el análisis sigue **adentro** del literal, y lo que viene es contenido de largo desconocido —texto, no código—. Descartarlo descarta contenido.
>
> Cuando un charLiteral falla, el análisis está en un punto donde el literal **ya debería haber terminado**: su contenido es exactamente un carácter. Lo que sigue no hay razón para creer que sea contenido del literal. Descartarlo descartaría código.

De ahí salen tres consecuencias concretas:

No hay nada que descartar. El modo pánico existe para saltear una corrida de basura de largo arbitrario. En un charLiteral esa corrida tiene largo a lo sumo uno: al detectar el error se consumieron la comilla de apertura y un carácter.

La comilla simple no es un delimitador confiable. En un litString la " de cierre es inequívoca, porque todo lo que está entre comillas es contenido. En un charLiteral, una ' posterior puede ser tanto el cierre de éste como la apertura del siguiente —y muchas veces no hay ninguna: en `'a; x = 1;` no aparece una segunda comilla en toda la línea, así que descartar hasta el cierre se comería cinco tokens legítimos.

La recuperación estructural ya alcanza. Como el literal es acotado, el criterio general —consumir exactamente lo que se reporta— basta por sí solo: al reportar `'a` la lectura quedó sobre el carácter ofensivo y el análisis retoma ahí sin ningún paso adicional.

El costo, asumido a propósito. Un `'abc'` produce *dos* errores y un idMV(bc) de más, en vez de un único error limpio:

```
'ab'; var x = 1;
  -> Error Léxico en línea 1, columna 1: charLiteral mal formado, contiene más de un carácter: 'a
     [Error:'a|1]
     (idMV,b,1)
     Error Léxico en línea 1, columna 4: charLiteral mal formado, contiene más de un carácter: ';
     [Error:';|1]
     (pr_var,var,1) (idMV,x,1) (op_asignacion,=,1) (intLiteral,1,1) (puntoYComa,;,1)
```

Es idéntico a lo que hace `javac`. No es una analogía: sobre el mismo caso, la recuperación de `javac` coincide token por token con la de este compilador. Su salida real, verificada ejecutándolo:

```
$ javac T.java
T.java:1: error: unclosed character literal
class T { void m() { char c = 'ab'; int x = 1; } }
                             ^
T.java:1: error: unclosed character literal
class T { void m() { char c = 'ab'; int x = 1; } }
                               ^
T.java:1: error: not a statement
class T { void m() { char c = 'ab'; int x = 1; } }
                                ^
3 errors
```

La correspondencia es exacta en las cuatro decisiones que definen la recuperación:

Decisión	`javac`	**este compilador**
Errores léxicos sobre `'ab'`	2	2
Dónde caen	en las dos comillas	en las dos comillas
Qué pasa con la `b` del medio	identificador	idMV(b)
Qué pasa con el código posterior	sobrevive entero	sobrevive entero

En `javac` la tercera línea —*not a statement*, con el ^ sobre la `b`— es ya un error del análisis sintáctico, y es justamente la prueba de que su léxico entregó la `b` como un identificador en vez de descartarla. La única diferencia está en el *texto* del diagnóstico, no en la recuperación: `javac` usa un mismo mensaje para los dos errores, mientras que acá se distingue *sin cerrar* —lo cortó un fin de línea o de archivo— de *mal formado* —contiene más de un carácter—, que son dos problemas distintos y ayudan distinto.

Por qué se acepta ese costo. Porque la alternativa es peor. Lo que se ve en la última línea del ejemplo es que todo el código posterior al literal roto sobrevive intacto; si se descartara hasta la comilla de cierre, se perdería. Se prefiere un error de más antes que un token legítimo de menos: un error extra el lector lo ignora, mientras que un token que desaparece arruina el diagnóstico de todo lo que viene después.

6.4 Dónde apunta el ^

Se adoptó el criterio de `javac`, verificado ejecutándolo sobre cada caso:

> Si el mensaje habla de un **carácter ofensivo puntual**, el ^ va a ese carácter.
> Si habla de la **construcción entera** que falló, el ^ va a donde esa construcción **empieza**.

6.5 Ubicar el error sin releer el archivo

El reporte necesita la columna y el texto completo de la línea del error. Ninguna de las dos cosas obliga a una segunda pasada sobre el fuente:

Operación	**Costo**	**Cómo**
columna	$O(1)$	contador que avanza al leer y se reinicia en cada salto de línea
texto de la línea	O(largo de la línea)	otro contador guarda dónde empieza la línea actual

La información de posición se construye *mientras* se lee, con dos enteros, y el recorrido para recuperar la línea está acotado por su largo, que es exactamente el texto que el reporte va a imprimir de todos modos.

7 Logros perseguidos

El proyecto aspira a **los ocho logros** de la Etapa 1, perseguidos desde el comienzo del desarrollo y no como agregados posteriores sobre un mínimo producto viable:

1. Floats I
2. Unicodes
3. Multi-detección de Errores Léxicos
4. Reporte de Error Elegante
5. Manejador de Archivos Eficiente
6. Imbatibilidad Léxica
7. Good Job! I
8. Entrega Anticipada Léxica

Cuatro de ellos —Floats I, Unicodes, Multi-detección y Reporte de Error Elegante— cambian el comportamiento observable del compilador. Para que tanto el comportamiento base que pide la consigna como el del logro sean demostrables, cada uno vive detrás de un interruptor de configuración, todos en `true` por omisión:

```
ConfiguracionDelCompilador(floatsHabilitados, unicodeHabilitado,
                           reporteElegante, multiDeteccionDeErrores)
```

Sin esos interruptores el comportamiento base quedaría sepultado bajo el del logro y no habría forma de mostrar los dos.

8 Estrategia de prueba

El catálogo de casos de test **es el propio directorio de casos** del repositorio, con el oráculo escrito en las primeras líneas de cada archivo en el formato que definen los testers provistos. Como esas líneas son comentarios MiniJava válidos, el archivo es a la vez el caso de prueba y su propia especificación: el compilador las ignora y el tester las lee.

```
resources/
    sinErrores/    75 archivos   oráculo: //#lexema,nroLinea
    conErrores/    81 archivos   oráculo: // [Error:lexema|nroLinea]
    regresion/                   entradas que rompieron una corrida de fuzzing
```

8.1 Cómo se declara el oráculo de un caso

Un caso de `sinErrores/` declara, con una línea `//#lexema,nroLinea` por token, la secuencia completa que debe salir. Un caso de `conErrores/` declara además, con `// [Error:lexema|nroLinea]`, cada código de error esperado.

El tester provisto verifica únicamente que el *primer* código de error declarado aparezca en algún lugar de la salida. Eso deja dos huecos: un caso con varios errores queda casi sin verificar, y nada comprueba qué tokens salen entre medio —que es justamente donde se esconderían los errores de recuperación—. Por eso la cabecera admite una marca extra:

`// [Completo]` — de esta línea en adelante, la secuencia declarada debe coincidir **exactamente** con la salida real: todos los códigos de error y todos los tokens, en orden y sin nada de más ni de menos.

Sin esa marca la verificación es de *subsecuencia*: los elementos declarados tienen que aparecer todos y en orden, pero se admite que haya otros entre medio. Con la marca es de igualdad. Un caso completo se ve así:

```
// [Error:"a\u12|6]     <- primera línea: el código que lee el tester provisto
// [Completo]           <- de acá en más, verificación exacta
// [Error:"a\u12|6]
//#+,6
//#v1,6
//#$,7
"a\u12G4" + v1
```

Se lee así: sobre ese fuente, el compilador debe emitir el error del escape Unicode mal formado con lexema `"a\u12` en la línea 6, después el token `+`, después `v1`, y cerrar con el EOF en la línea 7 — exactamente eso y nada más. La primera línea está duplicada a propósito: el tester provisto le hace `substring(3)` a la línea 1 del archivo, así que ahí tiene que estar sí o sí el código de error.

9 Divergencias deliberadas respecto de Java

Ninguna de éstas es un descuido: todas están cubiertas por casos de test escritos a propósito, y todas tienen su razón.

Caso	MiniJava	Por qué
`\uXXXX` fuera de un literal	en un comentario es texto común	en Java el escape se traduce antes de tokenizar, en todo el archivo; implementar ese pre-procesamiento es una fase entera aparte que la consigna no pide
`041`, `0x1F`, `0b1010`	decimal; no hay literales octales, hexadecimales ni binarios	la ER de la especificación es decimal y nada más
`1_000`	intLiteral(1), error por el `_`, intLiteral(000)	ídem: no hay separadores de miles
`0000000001`	error	la cota es de *dígitos*, no de valor como en Java
`1e999`	floatLiteral válido	el rango es un chequeo semántico, no léxico
`.5`	punto + intLiteral	la consigna exige que un float empiece con dígito
`'\q'`	charLiteral válido	permisividad explícita de la especificación
`"a\q"`	litString válido	la especificación no define un conjunto de escapes: define qué comilla cierra

Caso	MiniJava	Por qué
`"a\\"`	*sin cerrar*	la regla es **posicional**: la comilla de cierre nunca está antecedida por \, y una barra no cancela a la anterior
Dígitos arábe-índicos	son un intLiteral	`Character.isDigit` los acepta; restringir a ASCII contradiría el logro Unicodes
Un ideograma CJK (`U+4E2D`) al empezar un identificador	error, aunque sí puede continuarlo	la ER dice “letra mayúscula” / “letra minúscula”, y no es ninguna de las dos
Form feed y tabulador vertical	error léxico	no están en la lista de separadores
BOM al inicio del archivo	carácter desconocido	ídem